Jackett Management UI & System Database â€” Implementation Plan

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Ship a Go-canonical `boba-jackett` service on port 7189 with a SQLite system database (encrypted credentials), a full Angular Jackett management UI in the Ð‘Ð¾Ñ±Ð° dashboard, and reach the 13-item Definition-of-Done from the design spec with zero loose ends.

Architecture: New Go service in `qBitTorrent-go/cmd/boba-jackett` runs alongside the existing Python merge service. SQLite at `config/boba.db` is canonical for credentials/indexer state/catalog cache; `.env` is mirrored on every change; Jackett is configured live via its REST API. AES-256-GCM encrypts credential values at rest with `BOBA_MASTER_KEY` auto-generated into `.env` on first boot. Two new Angular routes (`/jackett/credentials`, `/jackett/indexers`) call the new Go service.

Tech Stack: Go 1.26 (existing), `github.com/mattn/go-sqlite3`, Go stdlib `crypto/aes` + `crypto/cipher`, `net/http` + `gorilla/mux` (matching existing Go service patterns), Angular 21 + Material (existing dashboard), `go-swagger/swag` for OpenAPI generation, `bandit`/`gosec` for security static-analysis (already in CI).

Spec: `docs/superpowers/specs/2026-04-27-jackett-management-ui-and-system-db-design.md`

Predecessor: `docs/superpowers/plans/2026-04-26-jackett-autoconfig-clean-rebuild.md` (Layers 1-7 of Python autoconfig â€” shipped, will be reconciled in Task 43)

Conventions Used Throughout

- **Resource limits on every test invocation** (CONST-09): prefix with `GOMAXPROCS=2 nice -n 19 ionice -c 3`.
- **Conventional Commits** with the body line `Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>`.
- **Local commits only** during implementation. Final push to all remotes happens in Task 53 (one batch).
- **Real services only** for non-unit tests (CONST-11). Use the live Jackett container; do NOT mock it past Layer 1.
- **Reproduction-before-fix** (CONST-32): if a bug surfaces, write a challenge that reproduces it FIRST, confirm fail, then fix, then re-run.
- **Concurrent-safe containers** (CONST-13): any new shared map/slice in Go uses `digital.vasic.concurrency/pkg/safe` or equivalent â€” never bare `sync.Mutex + map`.
- **No host power management** (CONST-33): the projectâ€™s two challenge scripts (`no_suspend_calls_challenge.sh` +

`host_no_auto_suspend_challenge.sh`) must pass after every commit batch. Already verified clean on 2026-04-27.

File Structure (locked-in decomposition)

New files to create

```
qBitTorrent-go/
â”œâ”€â”€ cmd/boba-jackett/
â”‚   â””â”€â”€ main.go                                  # Service entrypoi
â”œâ”€â”€ Dockerfile.jackett                          # New container imag
â”œâ”€â”€ internal/
â”‚   â”œâ”€â”€ db/
â”‚   â”‚   â”œâ”€â”€ conn.go                               # SQLite connect
â”‚   â”‚   â”œâ”€â”€ crypto.go                             # AES-256-GCM he
â”‚   â”‚   â”œâ”€â”€ crypto_test.go
â”‚   â”‚   â”œâ”€â”€ migrate.go                            # Embedded migra
â”‚   â”‚   â”œâ”€â”€ migrate_test.go
â”‚   â”‚   â”œâ”€â”€ migrations/
â”‚   â”‚   â”‚   â””â”€â”€ 001_init.sql                        # Full schema
â”‚   â”‚   â””â”€â”€ repos/
â”‚   â”‚       â”œâ”€â”€ credentials.go
â”‚   â”‚       â”œâ”€â”€ credentials_test.go
â”‚   â”‚       â”œâ”€â”€ indexers.go
â”‚   â”‚       â”œâ”€â”€ indexers_test.go
â”‚   â”‚       â”œâ”€â”€ catalog.go
â”‚   â”‚       â”œâ”€â”€ catalog_test.go
â”‚   â”‚       â”œâ”€â”€ runs.go
â”‚   â”‚       â”œâ”€â”€ runs_test.go
â”‚   â”‚       â”œâ”€â”€ overrides.go
â”‚   â”‚       â””â”€â”€ overrides_test.go
â”‚   â”œâ”€â”€ envfile/
â”‚   â”‚   â”œâ”€â”€ parse.go                              # .env reader (h
â”‚   â”‚   â”œâ”€â”€ parse_test.go
â”‚   â”‚   â”œâ”€â”€ write.go                              # Atomic write:
â”‚   â”‚   â””â”€â”€ write_test.go
â”‚   â”œâ”€â”€ bootstrap/
â”‚   â”‚   â”œâ”€â”€ bootstrap.go                          # Master key aut
â”‚   â”‚   â”œâ”€â”€ bootstrap_test.go
â”‚   â”‚   â””â”€â”€ import.go                             # .env triple â†
â”‚   â”œâ”€â”€ jackett/
â”‚   â”‚   â”œâ”€â”€ client.go                             # HTTP client (s
â”‚   â”‚   â”œâ”€â”€ client_test.go
â”‚   â”‚   â”œâ”€â”€ matcher.go                            # Fuzzy + overri
â”‚   â”‚   â”œâ”€â”€ matcher_test.go
â”‚   â”‚   â”œâ”€â”€ autoconfig.go                         # Orchestrator (
â”‚   â”‚   â””â”€â”€ autoconfig_test.go
â”‚   â”œâ”€â”€ jackettapi/
â”‚   â”‚   â”œâ”€â”€ auth_middleware.go                    # admin/admin se
â”‚   â”‚   â”œâ”€â”€ credentials.go                        # /api/v1/jacket
â”‚   â”‚   â”œâ”€â”€ credentials_test.go
â”‚   â”‚   â”œâ”€â”€ indexers.go                           # /api/v1/jacket
```

```
â”‚   â”‚   â”œâ”€â”€ indexers_test.go
â”‚   â”‚   â”œâ”€â”€ catalog.go                          # /api/v1/jacket
â”‚   â”‚   â”œâ”€â”€ catalog_test.go
â”‚   â”‚   â”œâ”€â”€ runs.go                             # /api/v1/jacket
â”‚   â”‚   â”œâ”€â”€ runs_test.go
â”‚   â”‚   â”œâ”€â”€ overrides.go                        # /api/v1/jacket
â”‚   â”‚   â”œâ”€â”€ overrides_test.go
â”‚   â”‚   â”œâ”€â”€ health.go                           # /healthz handl
â”‚   â”‚   â”œâ”€â”€ router.go                           # Mux setup with
â”‚   â”‚   â”œâ”€â”€ openapi.go                          # Generated Open
â”‚   â”‚   â””â”€â”€ logging/
â”‚   â”‚       â”œâ”€â”€ redactor.go                   # io.Writer wrappe
â”‚   â”‚       â””â”€â”€ redactor_test.go
â””â”€â”€ tests/
    â”œâ”€â”€ integration/
    â”‚   â””â”€â”€ jackett_db_test.go                # Real SQLite + re
    â”œâ”€â”€ e2e/
    â”‚   â””â”€â”€ jackett_management_test.go        # Real stack
    â”œâ”€â”€ security/
    â”‚   â””â”€â”€ credential_leak_test.go           # The leak-grep te
    â””â”€â”€ contract/
        â””â”€â”€ openapi_test.go                 # Validate live resp

frontend/src/app/
â”œâ”€â”€ jackett/
â”‚   â”œâ”€â”€ jackett.module.ts
â”‚   â”œâ”€â”€ jackett-routing.module.ts
â”‚   â”œâ”€â”€ credentials/
â”‚   â”‚   â”œâ”€â”€ credentials.component.ts
â”‚   â”‚   â”œâ”€â”€ credentials.component.html
â”‚   â”‚   â”œâ”€â”€ credentials.component.scss
â”‚   â”‚   â”œâ”€â”€ credential-edit-dialog.component.ts
â”‚   â”‚   â””â”€â”€ credentials.service.ts
â”‚   â””â”€â”€ indexers/
â”‚       â”œâ”€â”€ indexers.component.ts             # Tab container
â”‚       â”œâ”€â”€ indexers.component.html
â”‚       â”œâ”€â”€ configured-tab.component.ts
â”‚       â”œâ”€â”€ catalog-tab.component.ts
â”‚       â”œâ”€â”€ history-tab.component.ts
â”‚       â”œâ”€â”€ indexer-add-dialog.component.ts
â”‚       â”œâ”€â”€ iptorrents-cookie-flow.component.ts
â”‚       â””â”€â”€ indexers.service.ts

challenges/scripts/
â”œâ”€â”€ cred_roundtrip_challenge.sh
â”œâ”€â”€ env_db_drift_challenge.sh
â”œâ”€â”€ iptorrents_cookie_flow_challenge.sh
â”œâ”€â”€ master_key_autogen_challenge.sh
â”œâ”€â”€ credential_leak_grep_challenge.sh
â”œâ”€â”€ nnmclub_native_plugin_clarification_challenge.sh
â””â”€â”€ boba_db_file_perms_challenge.sh

docs/
```

```
â”œâ”€â”€ BOBA_DATABASE.md                                  # NEW: schema, maste
â””â”€â”€ issues/fixed/BUGFIXES.md                          # If not present, cr
```

Files to modify

```
qBitTorrent-go/go.mod                                 # Add go-sqlite3 dep
qBitTorrent-go/scripts/build.sh                       # Add boba-jackett to buil
docker-compose.yml                                    # Add boba-jackett service
start.sh                                              # Add ensure_boba_master_k
download-proxy/src/api/jackett.py                     # REMOVE (replaced by Go s
download-proxy/src/api/__init__.py                    # REMOVE jackett router in
frontend/src/app/app.routes.ts                        # Register /jackett route
frontend/src/app/app.html                             # Add Jackett nav entry
docs/JACKETT_INTEGRATION.md                           # Rewrite Â§"Auto-Configur
CLAUDE.md                                             # Update Architecture + Po
AGENTS.md                                             # Same updates as CLAUDE.m
docs/superpowers/plans/2026-04-26-jackett-autoconfig-clean-rebuild.md  # R
```

Phase 1 â€” Foundations (Tasks 1-9)

Goal: Working Go service skeleton with SQLite + crypto + envfile primitives, all unit-tested, no HTTP server yet.

Task 1: Bootstrap project skeleton

Files: - Create: `qBitTorrent-go/cmd/boba-jackett/main.go` - Modify: `qBitTorrent-go/go.mod` - Modify: `qBitTorrent-go/scripts/build.sh`

- [ ] **Step 1: Create `cmd/boba-jackett/main.go` minimal stub**

```
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Fprintln(os.Stderr, "boba-jackett: not yet implemented")
    os.Exit(1)
}
```

- [ ] **Step 2: Add SQLite driver dependency**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go get github.com/mattn/go-sqlite3@latest`

Expected: `go.mod` and `go.sum` updated.

- [ ]

- [ ] **Step 3: Update build script**

Edit `qBitTorrent-go/scripts/build.sh`, append after the `webui-bridge` build line:

```
echo "Building boba-jackett..."
cd "$ROOT_DIR" && go build -o "$BIN_DIR/boba-jackett" ./cmd/boba-jackett
```

- [ ] **Step 4: Verify build**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 bash scripts/build.sh`

Expected: three binaries in `bin/` (qbittorrent-proxy, webui-bridge, boba-jackett).

- [ ] **Step 5: Commit**

```
git add qBitTorrent-go/cmd/boba-jackett/main.go qBitTorrent-go/go.mod qBit
git commit -m "$(cat <<'EOF'
feat(boba-jackett): project skeleton + SQLite dep + build wiring

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Task 2: AES-256-GCM crypto helpers

Files: - Create: `qBitTorrent-go/internal/db/crypto.go` - Test: `qBitTorrent-go/internal/db/crypto_test.go`

- [ ] **Step 1: Write the failing test**

Create `internal/db/crypto_test.go`:

```
package db

import (
    "bytes"
    "crypto/rand"
    "strings"
    "testing"
)

func key32() []byte {
    k := make([]byte, 32)
    _, _ = rand.Read(k)
    return k
}

func TestEncryptDecryptRoundTrip(t *testing.T) {
```

```
    k := key32()
    plain := "rutracker_password_xyz!@#"
    blob, err := Encrypt(k, plain)
    if err != nil {
        t.Fatalf("Encrypt: %v", err)
    }
    got, err := Decrypt(k, blob)
    if err != nil {
        t.Fatalf("Decrypt: %v", err)
    }
    if got != plain {
        t.Fatalf("round-trip mismatch: got %q want %q", got, plain)
    }
}

func TestEncryptRejectsEmpty(t *testing.T) {
    if _, err := Encrypt(key32(), ""); err == nil {
        t.Fatal("expected error encrypting empty plaintext")
    }
}

func TestEncryptRejectsBadKey(t *testing.T) {
    if _, err := Encrypt(make([]byte, 16), "x"); err == nil {
        t.Fatal("expected error with 16-byte key")
    }
}

func TestDecryptDetectsTamper(t *testing.T) {
    k := key32()
    blob, _ := Encrypt(k, "secret")
    blob[len(blob)-1] ^= 0x01 // flip last bit
    if _, err := Decrypt(k, blob); err == nil {
        t.Fatal("expected GCM auth failure on tampered ciphertext")
    }
}

func TestNonceUniqueness(t *testing.T) {
    k := key32()
    seen := make(map[string]struct{}, 100000)
    for i := 0; i < 100000; i++ {
        blob, _ := Encrypt(k, "x")
        nonce := string(blob[:12])
        if _, dup := seen[nonce]; dup {
            t.Fatalf("nonce collision at iter %d", i)
        }
        seen[nonce] = struct{}{}
    }
}

func TestBlobShape(t *testing.T) {
    blob, _ := Encrypt(key32(), "x")
    if len(blob) < 12+16 { // nonce(12) + GCM tag(16) + at least 1 byte
        t.Fatalf("blob too short: %d", len(blob))
    }
```

```
    if !bytes.Equal(blob[:12], blob[:12]) || strings.Contains(string(blob)
        // just sanity â€” the test above already proves round-trip
    }
}
```

- ☐ **Step 2: Run test to verify FAIL**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short -run Crypto ./internal/db/ -v` Expected: FAIL â€” `Encrypt`/`Decrypt` undefined.

- ☐ **Step 3: Implement** `internal/db/crypto.go`

```
package db

import (
    "crypto/aes"
    "crypto/cipher"
    "crypto/rand"
    "errors"
    "fmt"
    "io"
)

const nonceSize = 12

var (
    ErrEmptyPlaintext = errors.New("crypto: empty plaintext")
    ErrShortBlob      = errors.New("crypto: blob shorter than nonce")
)

func Encrypt(key []byte, plaintext string) ([]byte, error) {
    if len(plaintext) == 0 {
        return nil, ErrEmptyPlaintext
    }
    block, err := aes.NewCipher(key)
    if err != nil {
        return nil, fmt.Errorf("aes.NewCipher: %w", err)
    }
    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, fmt.Errorf("cipher.NewGCM: %w", err)
    }
    nonce := make([]byte, nonceSize)
    if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
        return nil, fmt.Errorf("rand.Read: %w", err)
    }
    ct := gcm.Seal(nil, nonce, []byte(plaintext), nil)
    out := make([]byte, 0, nonceSize+len(ct))
    out = append(out, nonce...)
    out = append(out, ct...)
    return out, nil
}
```

```
func Decrypt(key []byte, blob []byte) (string, error) {
    if len(blob) < nonceSize {
        return "", ErrShortBlob
    }
    block, err := aes.NewCipher(key)
    if err != nil {
        return "", fmt.Errorf("aes.NewCipher: %w", err)
    }
    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return "", fmt.Errorf("cipher.NewGCM: %w", err)
    }
    nonce, ct := blob[:nonceSize], blob[nonceSize:]
    pt, err := gcm.Open(nil, nonce, ct, nil)
    if err != nil {
        return "", fmt.Errorf("gcm.Open: %w", err)
    }
    return string(pt), nil
}
```

- ☐ **Step 4: Run tests to verify PASS**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short -run Crypto ./internal/db/ -v` Expected: 5 PASS.

- ☐ **Step 5: Commit**

```
git add qBitTorrent-go/internal/db/crypto.go qBitTorrent-go/internal/db/cr
git commit -m "$(cat <<'EOF'
feat(boba-jackett): AES-256-GCM crypto helpers + tests

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Task 3: SQLite schema migration `001_init.sql` + stepper

Files: - Create: `qBitTorrent-go/internal/db/migrations/001_init.sql` - Create: `qBitTorrent-go/internal/db/migrate.go` - Create: `qBitTorrent-go/internal/db/conn.go` - Test: `qBitTorrent-go/internal/db/migrate_test.go`

- ☐ **Step 1: Create the migration SQL**

Create `internal/db/migrations/001_init.sql` with the exact contents from spec Â§5 (reproduced verbatim â€” paste from the spec, do not re-derive).

- ☐ **Step 2: Write the failing migrate test**

Create `internal/db/migrate_test.go`:

```
package db

import (
    "path/filepath"
    "testing"
)

func TestMigrateAppliesSchema(t *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "test.db")
    conn, err := Open(path)
    if err != nil {
        t.Fatalf("Open: %v", err)
    }
    defer conn.Close()
    if err := Migrate(conn); err != nil {
        t.Fatalf("Migrate: %v", err)
    }
    wantTables := []string{"credentials", "indexers", "catalog_cache", "au
    for _, tbl := range wantTables {
        var name string
        err := conn.QueryRow("SELECT name FROM sqlite_master WHERE type='t
        if err != nil {
            t.Errorf("table %s missing: %v", tbl, err)
        }
    }
}

func TestMigrateIdempotent(t *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "test.db")
    conn, _ := Open(path)
    defer conn.Close()
    if err := Migrate(conn); err != nil {
        t.Fatalf("first migrate: %v", err)
    }
    if err := Migrate(conn); err != nil {
        t.Fatalf("second migrate (should be no-op): %v", err)
    }
    var version int
    if err := conn.QueryRow("SELECT MAX(version) FROM schema_migrations").
        t.Fatalf("query version: %v", err)
    }
    if version != 1 {
        t.Fatalf("want version=1, got %d", version)
    }
}

func TestMigratePragmas(t *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "test.db")
    conn, _ := Open(path)
```

```
    defer conn.Close()
    _ = Migrate(conn)

    var jm string
    conn.QueryRow("PRAGMA journal_mode").Scan(&jm)
    if jm != "wal" {
        t.Errorf("journal_mode: want wal, got %s", jm)
    }
    var fk int
    conn.QueryRow("PRAGMA foreign_keys").Scan(&fk)
    if fk != 1 {
        t.Errorf("foreign_keys: want 1, got %d", fk)
    }
}
```

- [ ] **Step 3: Run test to verify FAIL**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short -run Migrate ./internal/db/ -v` Expected: FAIL (Open / Migrate undefined).

- [ ] **Step 4: Implement** `internal/db/conn.go`

```
package db

import (
    "database/sql"
    "fmt"

    _ "modernc.org/sqlite"
)

func Open(path string) (*sql.DB, error) {
    // modernc.org/sqlite uses _pragma= syntax for PRAGMA setup.
    dsn := fmt.Sprintf("file:%s?_pragma=journal_mode(WAL)&_pragma=foreign_
    conn, err := sql.Open("sqlite", dsn) // driver name "sqlite" (not "sql
    if err != nil {
        return nil, fmt.Errorf("sql.Open: %w", err)
    }
    conn.SetMaxOpenConns(1) // SQLite single-writer; reads also serialized
    if err := conn.Ping(); err != nil {
        conn.Close()
        return nil, fmt.Errorf("ping: %w", err)
    }
    return conn, nil
}
```

Driver-swap note (post-Task-1 review): Task 1â€™s commit `ddcf5f7` swapped `github.com/mattn/go-sqlite3` (CGO-only) for `modernc.org/sqlite` (pure Go). The blank import above locks the dep into the build graph so `go mod`

`tidy` no longer strips it. Driver name is `"sqlite"` (not `"sqlite3"`); DSN uses `_pragma=...(...)` syntax. All subsequent tasks reference this driver.

- ☐ **Step 5: Implement** `internal/db/migrate.go`

```
package db

import (
    "database/sql"
    "embed"
    "fmt"
    "sort"
    "strconv"
    "strings"
)

//go:embed migrations/*.sql
var migrationsFS embed.FS

func Migrate(conn *sql.DB) error {
    if _, err := conn.Exec(`CREATE TABLE IF NOT EXISTS schema_migrations (
        version INTEGER PRIMARY KEY,
        applied_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
    )`); err != nil {
        return fmt.Errorf("create schema_migrations: %w", err)
    }
    applied := map[int]bool{}
    rows, err := conn.Query("SELECT version FROM schema_migrations")
    if err != nil {
        return fmt.Errorf("query versions: %w", err)
    }
    for rows.Next() {
        var v int
        _ = rows.Scan(&v)
        applied[v] = true
    }
    rows.Close()

    entries, err := migrationsFS.ReadDir("migrations")
    if err != nil {
        return fmt.Errorf("read migrations dir: %w", err)
    }
    type mig struct {
        version int
        name    string
    }
    var migs []mig
    for _, e := range entries {
        name := e.Name()
        if !strings.HasSuffix(name, ".sql") {
            continue
        }
        verStr := strings.SplitN(name, "_", 2)[0]
```

```
        v, err := strconv.Atoi(verStr)
        if err != nil {
            return fmt.Errorf("bad migration name %s: %w", name, err)
        }
        migs = append(migs, mig{v, name})
    }
    sort.Slice(migs, func(i, j int) bool { return migs[i].version < migs[j

    for _, m := range migs {
        if applied[m.version] {
            continue
        }
        body, err := migrationsFS.ReadFile("migrations/" + m.name)
        if err != nil {
            return fmt.Errorf("read %s: %w", m.name, err)
        }
        tx, err := conn.Begin()
        if err != nil {
            return fmt.Errorf("begin tx for %s: %w", m.name, err)
        }
        if _, err := tx.Exec(string(body)); err != nil {
            tx.Rollback()
            return fmt.Errorf("apply %s: %w", m.name, err)
        }
        // 001_init.sql does its own schema_migrations INSERT.
        // Subsequent migrations rely on this stepper to track.
        if m.version > 1 {
            if _, err := tx.Exec("INSERT INTO schema_migrations(version) V
                tx.Rollback()
                return fmt.Errorf("record version %d: %w", m.version, err)
            }
        }
        if err := tx.Commit(); err != nil {
            return fmt.Errorf("commit %s: %w", m.name, err)
        }
    }
    return nil
}
```

- ☐ **Step 6: Run tests to verify PASS**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short -run Migrate ./internal/db/ -v` Expected: 3 PASS.

- ☐ **Step 7: Commit**

```
git add qBitTorrent-go/internal/db/
git commit -m "$(cat <<'EOF'
feat(boba-jackett): SQLite schema 001_init + migrations stepper + tests

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
```

```
EOF
)"
```

Task 4: envfile parser

Files: - Create: `qBitTorrent-go/internal/envfile/parse.go` - Test: `qBitTorrent-go/internal/envfile/parse_test.go`

- [ ] **Step 1: Write failing test**

```
package envfile

import (
    "reflect"
    "strings"
    "testing"
)

func TestParseAllForms(t *testing.T) {
    src := `# header comment
FOO=bar
QUOTED="hello world"
SINGLE='hi'
EMPTY=
SPACED  =  trimmed
# another comment
DUP=first
DUP=second
`
    got, err := Parse(strings.NewReader(src))
    if err != nil {
        t.Fatalf("Parse: %v", err)
    }
    want := map[string]string{
        "FOO": "bar", "QUOTED": "hello world", "SINGLE": "hi",
        "EMPTY": "", "SPACED": "trimmed", "DUP": "second",
    }
    if !reflect.DeepEqual(got, want) {
        t.Fatalf("got %+v want %+v", got, want)
    }
}

func TestParseHandlesBlanks(t *testing.T) {
    src := "\n\n\nFOO=bar\n\n"
    got, _ := Parse(strings.NewReader(src))
    if got["FOO"] != "bar" {
        t.Fatalf("got %+v", got)
    }
}
```

- [ ] **Step 2: Run â€” expect FAIL** (`Parse` undefined)

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short ./internal/envfile/ -v`

- [ ] **Step 3: Implement** `parse.go`

```
package envfile

import (
    "bufio"
    "io"
    "strings"
)

// Parse returns the LAST occurrence for duplicate keys (matches shell beh
func Parse(r io.Reader) (map[string]string, error) {
    out := map[string]string{}
    sc := bufio.NewScanner(r)
    for sc.Scan() {
        line := sc.Text()
        t := strings.TrimSpace(line)
        if t == "" || strings.HasPrefix(t, "#") {
            continue
        }
        eq := strings.IndexByte(t, '=')
        if eq < 0 {
            continue
        }
        k := strings.TrimSpace(t[:eq])
        v := strings.TrimSpace(t[eq+1:])
        // strip matching surrounding quotes
        if len(v) >= 2 {
            if (v[0] == '"' && v[len(v)-1] == '"') || (v[0] == '\'' && v[l
                v = v[1 : len(v)-1]
            }
        }
        out[k] = v
    }
    if err := sc.Err(); err != nil {
        return nil, err
    }
    return out, nil
}
```

- [ ] **Step 4: Run â€” expect 2 PASS**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short ./internal/envfile/ -v`

- [ ] **Step 5: Commit**

```
git add qBitTorrent-go/internal/envfile/parse.go qBitTorrent-go/internal/e
git commit -m "$(cat <<'EOF'
feat(boba-jackett): envfile parser handles comments/quotes/blanks/dups

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Task 5: envfile atomic writer

Files: - Create: qBitTorrent-go/internal/envfile/write.go - Test: qBitTorrent-go/internal/envfile/write_test.go

- ☐ **Step 1: Write failing tests**

```
package envfile

import (
    "os"
    "path/filepath"
    "strings"
    "sync"
    "testing"
)

func TestWriteRoundTrip(t *testing.T) {
    dir := t.TempDir()
    p := filepath.Join(dir, ".env")
    os.WriteFile(p, []byte("FOO=bar\n"), 0600)
    if err := Upsert(p, map[string]string{"BAZ": "qux", "FOO": "baz"}); er
        t.Fatalf("Upsert: %v", err)
    }
    body, _ := os.ReadFile(p)
    s := string(body)
    if !strings.Contains(s, "BAZ=qux") {
        t.Fatalf("missing BAZ=qux:\n%s", s)
    }
    if !strings.Contains(s, "FOO=baz") {
        t.Fatalf("missing FOO=baz:\n%s", s)
    }
    if strings.Contains(s, "FOO=bar") {
        t.Fatalf("old FOO=bar still present:\n%s", s)
    }
}

func TestWriteMode0600(t *testing.T) {
    dir := t.TempDir()
    p := filepath.Join(dir, ".env")
    os.WriteFile(p, []byte("FOO=bar\n"), 0644)
    _ = Upsert(p, map[string]string{"FOO": "baz"})
    st, _ := os.Stat(p)
    if st.Mode().Perm() != 0600 {
```

```
        t.Fatalf("want mode 0600, got %v", st.Mode().Perm())
    }
}

func TestDeleteKey(t *testing.T) {
    dir := t.TempDir()
    p := filepath.Join(dir, ".env")
    os.WriteFile(p, []byte("FOO=bar\nBAZ=qux\n"), 0600)
    if err := Delete(p, []string{"FOO"}); err != nil {
        t.Fatalf("Delete: %v", err)
    }
    body, _ := os.ReadFile(p)
    if strings.Contains(string(body), "FOO=") {
        t.Fatalf("FOO not deleted:\n%s", body)
    }
    if !strings.Contains(string(body), "BAZ=qux") {
        t.Fatalf("BAZ removed unexpectedly:\n%s", body)
    }
}

func TestConcurrentWritesNoCorruption(t *testing.T) {
    dir := t.TempDir()
    p := filepath.Join(dir, ".env")
    os.WriteFile(p, []byte(""), 0600)
    var wg sync.WaitGroup
    for i := 0; i < 50; i++ {
        i := i
        wg.Add(1)
        go func() {
            defer wg.Done()
            _ = Upsert(p, map[string]string{"K": "v"})
            _ = Upsert(p, map[string]string{"K2": "v2"})
            _ = i
        }()
    }
    wg.Wait()
    body, _ := os.ReadFile(p)
    got, err := Parse(strings.NewReader(string(body)))
    if err != nil {
        t.Fatalf("post-concurrent parse: %v\nbody:\n%s", err, body)
    }
    if got["K"] != "v" || got["K2"] != "v2" {
        t.Fatalf("post-concurrent values lost: %+v", got)
    }
}
```

- ☐ **Step 2: Run â€” expect FAIL**

- ☐ **Step 3: Implement** `write.go`

```
package envfile
```

```
import (
    "fmt"
    "os"
    "path/filepath"
    "sort"
    "strings"
    "sync"
)

var writerMu sync.Mutex

func Upsert(path string, kv map[string]string) error {
    writerMu.Lock()
    defer writerMu.Unlock()
    return mutate(path, func(lines []string) []string {
        seen := map[string]bool{}
        out := make([]string, 0, len(lines)+len(kv))
        for _, l := range lines {
            t := strings.TrimSpace(l)
            if t == "" || strings.HasPrefix(t, "#") {
                out = append(out, l)
                continue
            }
            eq := strings.IndexByte(t, '=')
            if eq < 0 {
                out = append(out, l)
                continue
            }
            k := strings.TrimSpace(t[:eq])
            if v, ok := kv[k]; ok {
                if seen[k] {
                    continue // drop duplicates
                }
                out = append(out, fmt.Sprintf("%s=%s", k, v))
                seen[k] = true
                continue
            }
            out = append(out, l)
        }
        // append new keys (sorted for determinism)
        newKeys := make([]string, 0)
        for k := range kv {
            if !seen[k] {
                newKeys = append(newKeys, k)
            }
        }
        sort.Strings(newKeys)
        for _, k := range newKeys {
            out = append(out, fmt.Sprintf("%s=%s", k, kv[k]))
        }
        return out
    })
}
```

```
func Delete(path string, keys []string) error {
    writerMu.Lock()
    defer writerMu.Unlock()
    drop := map[string]bool{}
    for _, k := range keys {
        drop[k] = true
    }
    return mutate(path, func(lines []string) []string {
        out := make([]string, 0, len(lines))
        for _, l := range lines {
            t := strings.TrimSpace(l)
            if t == "" || strings.HasPrefix(t, "#") {
                out = append(out, l)
                continue
            }
            eq := strings.IndexByte(t, '=')
            if eq < 0 {
                out = append(out, l)
                continue
            }
            k := strings.TrimSpace(t[:eq])
            if drop[k] {
                continue
            }
            out = append(out, l)
        }
        return out
    })
}

func mutate(path string, fn func([]string) []string) error {
    body, err := os.ReadFile(path)
    if err != nil && !os.IsNotExist(err) {
        return fmt.Errorf("read: %w", err)
    }
    lines := []string{}
    if len(body) > 0 {
        lines = strings.Split(strings.TrimRight(string(body), "\n"), "\n")
    }
    out := fn(lines)
    tmp := path + ".tmp"
    f, err := os.OpenFile(tmp, os.O_WRONLY|os.O_CREATE|os.O_TRUNC, 0600)
    if err != nil {
        return fmt.Errorf("open tmp: %w", err)
    }
    if _, err := f.WriteString(strings.Join(out, "\n") + "\n"); err != nil
        f.Close()
        os.Remove(tmp)
        return fmt.Errorf("write: %w", err)
    }
    if err := f.Sync(); err != nil {
        f.Close()
        os.Remove(tmp)
        return fmt.Errorf("fsync tmp: %w", err)
```

```
    }
    if err := f.Close(); err != nil {
        os.Remove(tmp)
        return fmt.Errorf("close tmp: %w", err)
    }
    if err := os.Chmod(tmp, 0600); err != nil {
        os.Remove(tmp)
        return fmt.Errorf("chmod tmp: %w", err)
    }
    if err := os.Rename(tmp, path); err != nil {
        os.Remove(tmp)
        return fmt.Errorf("rename: %w", err)
    }
    // fsync parent dir
    dir := filepath.Dir(path)
    d, err := os.Open(dir)
    if err == nil {
        _ = d.Sync()
        d.Close()
    }
    return nil
}
```

- ☐ **Step 4: Run â€" expect 4 PASS**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short ./internal/envfile/ -v`

- ☐ **Step 5: Commit**

```
git add qBitTorrent-go/internal/envfile/write.go qBitTorrent-go/internal/e
git commit -m "$(cat <<'EOF'
feat(boba-jackett): atomic envfile Upsert/Delete with mutex + fsync

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Task 6: Credentials repo (encrypted CRUD)

Files: - Create: `qBitTorrent-go/internal/db/repos/credentials.go` - Test: `qBitTorrent-go/internal/db/repos/credentials_test.go`

- ☐ **Step 1: Write failing tests**

```
package repos

import (
    "crypto/rand"
    "path/filepath"
```

```
    "testing"
    "time"

    "github.com/milos85vasic/qBitTorrent-go/internal/db"
)

func freshConn(t *testing.T) *Credentials {
    t.Helper()
    dir := t.TempDir()
    conn, err := db.Open(filepath.Join(dir, "t.db"))
    if err != nil {
        t.Fatalf("Open: %v", err)
    }
    if err := db.Migrate(conn); err != nil {
        t.Fatalf("Migrate: %v", err)
    }
    key := make([]byte, 32)
    rand.Read(key)
    return NewCredentials(conn, key)
}

func TestUpsertAndGet(t *testing.T) {
    r := freshConn(t)
    if err := r.Upsert("RUTRACKER", "userpass", strPtr("u"), strPtr("p"),
        t.Fatalf("Upsert: %v", err)
    }
    got, err := r.Get("RUTRACKER")
    if err != nil {
        t.Fatalf("Get: %v", err)
    }
    if got.Name != "RUTRACKER" || got.Kind != "userpass" {
        t.Fatalf("bad row: %+v", got)
    }
    if got.Username != "u" || got.Password != "p" {
        t.Fatalf("decrypt mismatch: %+v", got)
    }
}

func TestList(t *testing.T) {
    r := freshConn(t)
    r.Upsert("RUTRACKER", "userpass", strPtr("u"), strPtr("p"), nil)
    r.Upsert("IPTORRENTS", "cookie", nil, nil, strPtr("c"))
    rows, err := r.List()
    if err != nil {
        t.Fatalf("List: %v", err)
    }
    if len(rows) != 2 {
        t.Fatalf("want 2 rows, got %d", len(rows))
    }
}

func TestDelete(t *testing.T) {
    r := freshConn(t)
    r.Upsert("X", "userpass", strPtr("u"), strPtr("p"), nil)
```

```
    if err := r.Delete("X"); err != nil {
        t.Fatalf("Delete: %v", err)
    }
    _, err := r.Get("X")
    if err == nil {
        t.Fatal("expected not-found")
    }
}

func TestUpsertOverwritesUpdatedAt(t *testing.T) {
    r := freshConn(t)
    r.Upsert("X", "userpass", strPtr("u"), strPtr("p"), nil)
    first, _ := r.Get("X")
    time.Sleep(1100 * time.Millisecond)
    r.Upsert("X", "userpass", strPtr("u2"), strPtr("p2"), nil)
    second, _ := r.Get("X")
    if !second.UpdatedAt.After(first.UpdatedAt) {
        t.Fatalf("updated_at did not advance: %v then %v", first.UpdatedAt
    }
}

func strPtr(s string) *string { return &s }
```

- ☐ **Step 2: Run â€” expect FAIL**

- ☐ **Step 3: Implement** `internal/db/repos/credentials.go`

```
package repos

import (
    "database/sql"
    "errors"
    "fmt"
    "time"

    "github.com/milos85vasic/qBitTorrent-go/internal/db"
)

type Credential struct {
    Name        string
    Kind        string
    Username    string
    Password    string
    Cookies     string
    HasUsername bool
    HasPassword bool
    HasCookies  bool
    CreatedAt   time.Time
    UpdatedAt   time.Time
    LastUsedAt  *time.Time
}
```

```
type Credentials struct {
    conn *sql.DB
    key  []byte
}

var ErrNotFound = errors.New("repos: credential not found")

func NewCredentials(conn *sql.DB, key []byte) *Credentials {
    return &Credentials{conn: conn, key: key}
}

func (c *Credentials) Upsert(name, kind string, user, pass, cookies *strin
    var (
        uEnc, pEnc, cEnc []byte
        err              error
    )
    if user != nil && *user != "" {
        uEnc, err = db.Encrypt(c.key, *user)
        if err != nil {
            return fmt.Errorf("encrypt username: %w", err)
        }
    }
    if pass != nil && *pass != "" {
        pEnc, err = db.Encrypt(c.key, *pass)
        if err != nil {
            return fmt.Errorf("encrypt password: %w", err)
        }
    }
    if cookies != nil && *cookies != "" {
        cEnc, err = db.Encrypt(c.key, *cookies)
        if err != nil {
            return fmt.Errorf("encrypt cookies: %w", err)
        }
    }
    now := time.Now().UTC()
    _, err = c.conn.Exec(`
        INSERT INTO credentials(name, kind, username_enc, password_enc, co
        VALUES(?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(name) DO UPDATE SET
            kind=excluded.kind,
            username_enc=COALESCE(excluded.username_enc, username_enc),
            password_enc=COALESCE(excluded.password_enc, password_enc),
            cookies_enc=COALESCE(excluded.cookies_enc, cookies_enc),
            updated_at=excluded.updated_at
    `, name, kind, uEnc, pEnc, cEnc, now, now)
    return err
}

func (c *Credentials) Get(name string) (*Credential, error) {
    row := c.conn.QueryRow(`SELECT name, kind, username_enc, password_enc,
        created_at, updated_at, last_used_at FROM credentials WHERE name=?
    cr := &Credential{}
    var uEnc, pEnc, cEnc []byte
    if err := row.Scan(&cr.Name, &cr.Kind, &uEnc, &pEnc, &cEnc,
```

```
        &cr.CreatedAt, &cr.UpdatedAt, &cr.LastUsedAt); err != nil {
        if errors.Is(err, sql.ErrNoRows) {
            return nil, ErrNotFound
        }
        return nil, fmt.Errorf("scan: %w", err)
    }
    if uEnc != nil {
        s, err := db.Decrypt(c.key, uEnc)
        if err != nil {
            return nil, fmt.Errorf("decrypt username: %w", err)
        }
        cr.Username = s
        cr.HasUsername = true
    }
    if pEnc != nil {
        s, err := db.Decrypt(c.key, pEnc)
        if err != nil {
            return nil, fmt.Errorf("decrypt password: %w", err)
        }
        cr.Password = s
        cr.HasPassword = true
    }
    if cEnc != nil {
        s, err := db.Decrypt(c.key, cEnc)
        if err != nil {
            return nil, fmt.Errorf("decrypt cookies: %w", err)
        }
        cr.Cookies = s
        cr.HasCookies = true
    }
    return cr, nil
}

func (c *Credentials) List() ([]*Credential, error) {
    rows, err := c.conn.Query(`SELECT name, kind, username_enc IS NOT NULL
        password_enc IS NOT NULL, cookies_enc IS NOT NULL,
        created_at, updated_at, last_used_at FROM credentials ORDER BY nam
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var out []*Credential
    for rows.Next() {
        cr := &Credential{}
        if err := rows.Scan(&cr.Name, &cr.Kind, &cr.HasUsername, &cr.HasPa
            &cr.CreatedAt, &cr.UpdatedAt, &cr.LastUsedAt); err != nil {
            return nil, err
        }
        out = append(out, cr)
    }
    return out, rows.Err()
}

func (c *Credentials) Delete(name string) error {
```

```
    _, err := c.conn.Exec("DELETE FROM credentials WHERE name=?", name)
    return err
}

func (c *Credentials) MarkUsed(name string) error {
    _, err := c.conn.Exec("UPDATE credentials SET last_used_at=CURRENT_TIM
    return err
}
```

- [ ] **Step 4: Run â€” expect 4 PASS**

Run: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -short ./internal/db/repos/ -run Credential -v`

- [ ] **Step 5: Commit**

```
git add qBitTorrent-go/internal/db/repos/credentials.go qBitTorrent-go/int
git commit -m "$(cat <<'EOF'
feat(boba-jackett): credentials repo with encrypted CRUD + tests

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Task 7: Indexers repo

Files: - Create: `qBitTorrent-go/internal/db/repos/indexers.go` - Test: `qBitTorrent-go/internal/db/repos/indexers_test.go`

- [ ] **Step 1: Write failing tests**

```
package repos

import (
    "path/filepath"
    "testing"

    "github.com/milos85vasic/qBitTorrent-go/internal/db"
)

func indexersConn(t *testing.T) *Indexers {
    t.Helper()
    dir := t.TempDir()
    conn, _ := db.Open(filepath.Join(dir, "t.db"))
    _ = db.Migrate(conn)
    return NewIndexers(conn)
}

func TestIndexersUpsertAndList(t *testing.T) {
    r := indexersConn(t)
```

```
    if err := r.Upsert(&Indexer{
        ID: "rutracker", DisplayName: "RuTracker.org", Type: "private",
        ConfiguredAtJackett: true, EnabledForSearch: true,
    }); err != nil {
        t.Fatalf("Upsert: %v", err)
    }
    rows, err := r.List()
    if err != nil {
        t.Fatalf("List: %v", err)
    }
    if len(rows) != 1 || rows[0].ID != "rutracker" {
        t.Fatalf("got %+v", rows)
    }
}

func TestIndexersDelete(t *testing.T) {
    r := indexersConn(t)
    r.Upsert(&Indexer{ID: "x", DisplayName: "X", Type: "public", EnabledFo
    if err := r.Delete("x"); err != nil {
        t.Fatalf("Delete: %v", err)
    }
    rows, _ := r.List()
    if len(rows) != 0 {
        t.Fatalf("want 0, got %d", len(rows))
    }
}

func TestIndexersToggleEnabled(t *testing.T) {
    r := indexersConn(t)
    r.Upsert(&Indexer{ID: "x", DisplayName: "X", Type: "public", EnabledFo
    if err := r.SetEnabled("x", false); err != nil {
        t.Fatalf("SetEnabled: %v", err)
    }
    rows, _ := r.List()
    if rows[0].EnabledForSearch {
        t.Fatal("expected disabled")
    }
}
```

- ☐ **Step 2: Run â€” expect FAIL**

- ☐ **Step 3: Implement** `internal/db/repos/indexers.go`

```
package repos

import (
    "database/sql"
    "time"
)

type Indexer struct {
    ID                  string
```

```
    DisplayName          string
    Type                 string
    ConfiguredAtJackett  bool
    LinkedCredentialName *string
    EnabledForSearch     bool
    LastJackettSyncAt    *time.Time
    LastTestStatus       *string
    LastTestAt           *time.Time
}

type Indexers struct{ conn *sql.DB }

func NewIndexers(conn *sql.DB) *Indexers { return &Indexers{conn: conn} }

func (r *Indexers) Upsert(i *Indexer) error {
    _, err := r.conn.Exec(`
        INSERT INTO indexers(id, display_name, type, configured_at_jackett
            linked_credential_name, enabled_for_search, last_jackett_sync_
            last_test_status, last_test_at)
        VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(id) DO UPDATE SET
            display_name=excluded.display_name,
            type=excluded.type,
            configured_at_jackett=excluded.configured_at_jackett,
            linked_credential_name=excluded.linked_credential_name,
            enabled_for_search=excluded.enabled_for_search,
            last_jackett_sync_at=excluded.last_jackett_sync_at,
            last_test_status=excluded.last_test_status,
            last_test_at=excluded.last_test_at
    `, i.ID, i.DisplayName, i.Type, i.ConfiguredAtJackett,
        i.LinkedCredentialName, i.EnabledForSearch, i.LastJackettSyncAt,
        i.LastTestStatus, i.LastTestAt)
    return err
}

func (r *Indexers) List() ([]*Indexer, error) {
    rows, err := r.conn.Query(`SELECT id, display_name, type, configured_a
        linked_credential_name, enabled_for_search, last_jackett_sync_at,
        last_test_status, last_test_at FROM indexers ORDER BY id`)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var out []*Indexer
    for rows.Next() {
        i := &Indexer{}
        if err := rows.Scan(&i.ID, &i.DisplayName, &i.Type, &i.ConfiguredA
            &i.LinkedCredentialName, &i.EnabledForSearch, &i.LastJackettSy
            &i.LastTestStatus, &i.LastTestAt); err != nil {
            return nil, err
        }
        out = append(out, i)
    }
    return out, rows.Err()
```

```
}

func (r *Indexers) Delete(id string) error {
    _, err := r.conn.Exec("DELETE FROM indexers WHERE id=?", id)
    return err
}

func (r *Indexers) SetEnabled(id string, enabled bool) error {
    _, err := r.conn.Exec("UPDATE indexers SET enabled_for_search=? WHERE
    return err
}

func (r *Indexers) RecordTest(id, status string) error {
    now := time.Now().UTC()
    _, err := r.conn.Exec("UPDATE indexers SET last_test_status=?, last_te
    return err
}
```

- ☐ **Step 4: Run â€” expect 3 PASS**
- ☐ **Step 5: Commit**

```
git add qBitTorrent-go/internal/db/repos/indexers.go qBitTorrent-go/intern
git commit -m "$(cat <<'EOF'
feat(boba-jackett): indexers repo CRUD + enable toggle + tests

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Task 8: Catalog cache repo

Files: - Create: `qBitTorrent-go/internal/db/repos/catalog.go` - Test: `qBitTorrent-go/internal/db/repos/catalog_test.go`

- ☐ **Step 1: Write failing tests** â€” minimal CRUD: `Upsert`, `Get`, `Query(filter)`, `OlderThan(d) []ID`. Tests follow the same pattern as Task 7.
- ☐ **Step 2: Run â€” expect FAIL**
- ☐ **Step 3: Implement** `catalog.go` with fields matching `catalog_cache` schema. Query supports `search` (LIKE on display_name+description), `type=`, `language=`, paginated `LIMIT ?, OFFSET ?`. Use prepared statements.
- ☐ **Step 4: Run â€” expect PASS**
- ☐

- **Step 5: Commit**: `feat(boba-jackett): catalog_cache repo with paginated query`

Task 9: Autoconfig runs repo + overrides repo

Two related repos in one task â€” both are simple CRUD.

Files: - Create: `qBitTorrent-go/internal/db/repos/runs.go` - Test: `qBitTorrent-go/internal/db/repos/runs_test.go` - Create: `qBitTorrent-go/internal/db/repos/overrides.go` - Test: `qBitTorrent-go/internal/db/repos/overrides_test.go`

- [ ] **Step 1: Tests for runs**: `Insert(result)`, `List(limit)`, `Get(id)` returning summary blob.
- [ ] **Step 2: Tests for overrides**: `List()`, `Upsert(env_name, indexer_id)`, `Delete(env_name)`.
- [ ] **Step 3: Run â€” expect FAIL**
- [ ] **Step 4: Implement both files following the same `*sql.DB` constructor pattern as Tasks 6/7**.
- [ ] **Step 5: Run â€” expect PASS**
- [ ] **Step 6: Commit**: `feat(boba-jackett): autoconfig_runs + overrides repos`

Phase 2 â€” Bootstrap & Jackett port (Tasks 10-14)

Task 10: Bootstrap (master key autogen + import)

Files: - Create: `qBitTorrent-go/internal/bootstrap/bootstrap.go` - Create: `qBitTorrent-go/internal/bootstrap/import.go` - Test: `qBitTorrent-go/internal/bootstrap/bootstrap_test.go`

- [ ] **Step 1: Write failing tests**

```
package bootstrap

import (
    "os"
    "path/filepath"
    "strings"
    "testing"

    "github.com/milos85vasic/qBitTorrent-go/internal/envfile"
)
```

```
func TestEnsureMasterKeyGeneratesIfMissing(t *testing.T) {
    dir := t.TempDir()
    envP := filepath.Join(dir, ".env")
    os.WriteFile(envP, []byte("FOO=bar\n"), 0600)
    key, generated, err := EnsureMasterKey(envP)
    if err != nil {
        t.Fatalf("EnsureMasterKey: %v", err)
    }
    if !generated {
        t.Fatal("expected generated=true")
    }
    if len(key) != 32 {
        t.Fatalf("want 32 bytes, got %d", len(key))
    }
    body, _ := os.ReadFile(envP)
    if !strings.Contains(string(body), "BOBA_MASTER_KEY=") {
        t.Fatalf("key not persisted to .env:\n%s", body)
    }
    if !strings.Contains(string(body), "DO NOT LOSE") {
        t.Fatalf("warning header missing:\n%s", body)
    }
}

func TestEnsureMasterKeyIdempotent(t *testing.T) {
    dir := t.TempDir()
    envP := filepath.Join(dir, ".env")
    os.WriteFile(envP, []byte("BOBA_MASTER_KEY=00112233445566778899aabbccd
    _, generated, _ := EnsureMasterKey(envP)
    if generated {
        t.Fatal("expected generated=false")
    }
}

func TestImportFromEnvBundlesTriples(t *testing.T) {
    src, _ := envfile.Parse(strings.NewReader(`RUTRACKER_USERNAME=u
RUTRACKER_PASSWORD=p
IPTORRENTS_COOKIES=c
INCOMPLETE_USERNAME=onlyuser
JACKETT_API_KEY=ignored
`))
    bundles := DiscoverCredentialBundles(src, defaultExclude())
    if len(bundles) != 2 {
        t.Fatalf("want 2 (RUTRACKER, IPTORRENTS), got %+v", bundles)
    }
}
```

- ☐ **Step 2: Run â€” expect FAIL**

- ☐ **Step 3: Implement** `bootstrap.go`

```
package bootstrap

import (
    "crypto/rand"
    "encoding/hex"
    "fmt"
    "os"

    "github.com/milos85vasic/qBitTorrent-go/internal/envfile"
)

const masterKeyHeader = `# === BOBA SYSTEM ===
# Master key for credential encryption-at-rest in config/boba.db.
# DO NOT LOSE THIS â€” credentials become unrecoverable without it.
# To rotate: see docs/BOBA_DATABASE.md Â§ "Key Rotation".`

func EnsureMasterKey(envPath string) (key []byte, generated bool, err erro
    f, err := os.Open(envPath)
    if err != nil {
        return nil, false, fmt.Errorf("open .env: %w", err)
    }
    parsed, err := envfile.Parse(f)
    f.Close()
    if err != nil {
        return nil, false, fmt.Errorf("parse .env: %w", err)
    }
    if existing, ok := parsed["BOBA_MASTER_KEY"]; ok && len(existing) == 6
        k, err := hex.DecodeString(existing)
        if err == nil && len(k) == 32 {
            return k, false, nil
        }
    }
    raw := make([]byte, 32)
    if _, err := rand.Read(raw); err != nil {
        return nil, false, fmt.Errorf("rand.Read: %w", err)
    }
    hexKey := hex.EncodeToString(raw)
    // Append header (as comments) then key
    body, err := os.ReadFile(envPath)
    if err != nil {
        return nil, false, fmt.Errorf("read .env: %w", err)
    }
    out := string(body)
    if len(out) > 0 && out[len(out)-1] != '\n' {
        out += "\n"
    }
    out += "\n" + masterKeyHeader + "\n"
    if err := os.WriteFile(envPath+".tmp", []byte(out), 0600); err != nil
        return nil, false, fmt.Errorf("write tmp: %w", err)
    }
    if err := os.Rename(envPath+".tmp", envPath); err != nil {
        return nil, false, fmt.Errorf("rename: %w", err)
    }
    if err := envfile.Upsert(envPath, map[string]string{"BOBA_MASTER_KEY":
```

```
        return nil, false, fmt.Errorf("upsert key: %w", err)
    }
    return raw, true, nil
}

var defaultExcludeSet = map[string]bool{
    "QBITTORRENT": true, "JACKETT": true, "WEBUI": true,
    "PROXY": true, "MERGE": true, "BRIDGE": true, "BOBA": true,
}

func defaultExclude() map[string]bool {
    out := make(map[string]bool, len(defaultExcludeSet))
    for k, v := range defaultExcludeSet {
        out[k] = v
    }
    return out
}
```

- ☐ **Step 4: Implement** `import.go`

```
package bootstrap

import "regexp"

var credRE = regexp.MustCompile(`^([A-Z][A-Z0-9_]+?)_(USERNAME|PASSWORD|CO

type CredBundle struct {
    Name     string
    Username string
    Password string
    Cookies  string
}

func DiscoverCredentialBundles(env map[string]string, exclude map[string]b
    groups := map[string]*CredBundle{}
    for k, v := range env {
        m := credRE.FindStringSubmatch(k)
        if m == nil {
            continue
        }
        name, kind := m[1], m[2]
        if exclude[name] {
            continue
        }
        b := groups[name]
        if b == nil {
            b = &CredBundle{Name: name}
            groups[name] = b
        }
        switch kind {
        case "USERNAME":
            b.Username = v
        case "PASSWORD":
```

```
            b.Password = v
        case "COOKIES":
            b.Cookies = v
        }
    }
    var out []*CredBundle
    for _, b := range groups {
        hasUP := b.Username != "" && b.Password != ""
        hasC := b.Cookies != ""
        if hasUP || hasC {
            out = append(out, b)
        }
    }
    return out
}
```

- [ ] **Step 5: Run â€” expect 3 PASS**

- [ ] **Step 6: Commit**: `feat(boba-jackett): bootstrap (master key autogen + .env import)`

Task 11: Jackett HTTP client (warmup + catalog)

Files: - Create: `qBitTorrent-go/internal/jackett/client.go` - Test: `qBitTorrent-go/internal/jackett/client_test.go`

- [ ] **Step 1: Write failing tests using** `httptest.NewServer`

```
package jackett

import (
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
)

func TestSessionWarmupAndCatalog(t *testing.T) {
    warmedUp := false
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter,
        switch {
        case r.Method == "POST" && r.URL.Path == "/UI/Dashboard":
            http.SetCookie(w, &http.Cookie{Name: "Jackett", Value: "sessio
            warmedUp = true
            w.WriteHeader(302)
        case r.URL.Path == "/api/v2.0/indexers":
            if !warmedUp {
                w.WriteHeader(401)
                return
            }
```

```
            w.Header().Set("Content-Type", "application/json")
            w.Write([]byte(`[{"id":"rutracker","name":"RuTracker.org","typ
        }
    }))
    defer srv.Close()
    c := NewClient(srv.URL, "test-key")
    if err := c.WarmUp(); err != nil {
        t.Fatalf("WarmUp: %v", err)
    }
    cat, err := c.GetCatalog()
    if err != nil {
        t.Fatalf("GetCatalog: %v", err)
    }
    if len(cat) != 1 || cat[0].ID != "rutracker" {
        t.Fatalf("got %+v", cat)
    }
}

func TestPostConfig(t *testing.T) {
    var captured string
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter,
        if r.Method == "POST" && strings.Contains(r.URL.Path, "/config") {
            body := make([]byte, 1024)
            n, _ := r.Body.Read(body)
            captured = string(body[:n])
            w.WriteHeader(200)
        }
    }))
    defer srv.Close()
    c := NewClient(srv.URL, "k")
    body := []map[string]any{{"id": "username", "value": "u"}}
    if err := c.PostIndexerConfig("x", body); err != nil {
        t.Fatalf("Post: %v", err)
    }
    if !strings.Contains(captured, `"username"`) {
        t.Fatalf("body not posted: %s", captured)
    }
}
```

- ☐ **Step 2: Run â€” expect FAIL**

- ☐ **Step 3: Implement** `client.go`

```
package jackett

import (
    "bytes"
    "encoding/json"
    "fmt"
    "net/http"
    "net/http/cookiejar"
    "net/url"
```

```
    "strings"
    "time"
)

type CatalogEntry struct {
    ID         string                 `json:"id"`
    Name       string                 `json:"name"`
    Type       string                 `json:"type"`
    Configured bool                   `json:"configured"`
    Language   string                 `json:"language"`
    Description string                `json:"description"`
}

type Client struct {
    base   string
    apiKey string
    http   *http.Client
}

func NewClient(base, apiKey string) *Client {
    jar, _ := cookiejar.New(nil)
    return &Client{
        base:   strings.TrimRight(base, "/"),
        apiKey: apiKey,
        http:   &http.Client{Timeout: 30 * time.Second, Jar: jar},
    }
}

func (c *Client) WarmUp() error {
    form := url.Values{"password": {""}}
    req, _ := http.NewRequest("POST", c.base+"/UI/Dashboard", strings.NewR
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")
    resp, err := c.http.Do(req)
    if err != nil {
        return fmt.Errorf("warmup: %w", err)
    }
    defer resp.Body.Close()
    return nil
}

func (c *Client) GetCatalog() ([]CatalogEntry, error) {
    u, _ := url.Parse(c.base + "/api/v2.0/indexers")
    q := u.Query()
    q.Set("apikey", c.apiKey)
    q.Set("configured", "false")
    u.RawQuery = q.Encode()
    req, _ := http.NewRequest("GET", u.String(), nil)
    req.Header.Set("Accept", "application/json")
    resp, err := c.http.Do(req)
    if err != nil {
        return nil, fmt.Errorf("get catalog: %w", err)
    }
    defer resp.Body.Close()
    if resp.StatusCode == 401 {
```

```
        return nil, fmt.Errorf("jackett_auth_failed")
    }
    if resp.StatusCode >= 400 {
        return nil, fmt.Errorf("jackett_catalog_http_%d", resp.StatusCode)
    }
    var entries []CatalogEntry
    if err := json.NewDecoder(resp.Body).Decode(&entries); err != nil {
        return nil, fmt.Errorf("decode: %w", err)
    }
    return entries, nil
}

func (c *Client) GetIndexerTemplate(id string) ([]map[string]any, error) {
    u := fmt.Sprintf("%s/api/v2.0/indexers/%s/config?apikey=%s", c.base, i
    req, _ := http.NewRequest("GET", u, nil)
    req.Header.Set("Accept", "application/json")
    resp, err := c.http.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    if resp.StatusCode >= 400 {
        return nil, fmt.Errorf("template fetch HTTP %d", resp.StatusCode)
    }
    var raw any
    if err := json.NewDecoder(resp.Body).Decode(&raw); err != nil {
        return nil, err
    }
    switch v := raw.(type) {
    case []any:
        out := make([]map[string]any, 0, len(v))
        for _, item := range v {
            if m, ok := item.(map[string]any); ok {
                out = append(out, m)
            }
        }
        return out, nil
    case map[string]any:
        if cfg, ok := v["config"].([]any); ok {
            out := make([]map[string]any, 0, len(cfg))
            for _, item := range cfg {
                if m, ok := item.(map[string]any); ok {
                    out = append(out, m)
                }
            }
            return out, nil
        }
    }
    return nil, fmt.Errorf("unexpected template shape")
}

func (c *Client) PostIndexerConfig(id string, fields []map[string]any) err
    u := fmt.Sprintf("%s/api/v2.0/indexers/%s/config?apikey=%s", c.base, i
    body, _ := json.Marshal(fields)
```

```
    req, _ := http.NewRequest("POST", u, bytes.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    resp, err := c.http.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode >= 400 {
        return fmt.Errorf("config POST HTTP %d", resp.StatusCode)
    }
    return nil
}

func (c *Client) DeleteIndexer(id string) error {
    u := fmt.Sprintf("%s/api/v2.0/indexers/%s?apikey=%s", c.base, id, c.ap
    req, _ := http.NewRequest("DELETE", u, nil)
    resp, err := c.http.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode >= 400 && resp.StatusCode != 404 {
        return fmt.Errorf("delete HTTP %d", resp.StatusCode)
    }
    return nil
}
```

- ☐ **Step 4: Run â€” expect 2 PASS**
- ☐ **Step 5: Commit**: `feat(boba-jackett): Jackett HTTP client (warmup + catalog + config + delete)`

Task 12: Matcher (port from Python)

Files: - Create: `qBitTorrent-go/internal/jackett/matcher.go` - Test: `qBitTorrent-go/internal/jackett/matcher_test.go`

- ☐ **Step 1: Port Python tests verbatim** from `tests/unit/merge_service/test_jackett_autoconfig.py` â€” find all `_match_indexers` cases, translate to Go test functions covering: exact override, fuzzy match above threshold, ambiguous match, no match.
- ☐ **Step 2: Run â€” expect FAIL**
- ☐ **Step 3: Implement matcher**

Add `github.com/agnivade/levenshtein` dep:

```
cd qBitTorrent-go && go get github.com/agnivade/levenshtein
```

```
package jackett

import (
    "sort"
    "strings"

    "github.com/agnivade/levenshtein"
)

const FuzzyThreshold = 0.85

type AmbiguousMatch struct {
    EnvName    string
    Candidates []string
}

func ratio(a, b string) float64 {
    if a == "" && b == "" {
        return 1.0
    }
    d := levenshtein.ComputeDistance(a, b)
    maxLen := len(a)
    if len(b) > maxLen {
        maxLen = len(b)
    }
    if maxLen == 0 {
        return 1.0
    }
    return 1.0 - float64(d)/float64(maxLen)
}

func MatchIndexers(envNames []string, catalog []CatalogEntry, override map
    matched = map[string]string{}
    ids := map[string]bool{}
    for _, e := range catalog {
        ids[e.ID] = true
    }
    for _, name := range envNames {
        if t, ok := override[name]; ok && ids[t] {
            matched[name] = t
            continue
        }
        needle := strings.ToLower(name)
        type scored struct {
            id    string
            score float64
        }
        var scoredAll []scored
        for _, e := range catalog {
            s := ratio(needle, strings.ToLower(e.ID))
            if n := ratio(needle, strings.ToLower(e.Name)); n > s {
                s = n
            }
            if s >= FuzzyThreshold {
```

```
                scoredAll = append(scoredAll, scored{e.ID, s})
            }
        }
        if len(scoredAll) == 0 {
            unmatched = append(unmatched, name)
            continue
        }
        sort.Slice(scoredAll, func(i, j int) bool {
            if scoredAll[i].score != scoredAll[j].score {
                return scoredAll[i].score > scoredAll[j].score
            }
            return scoredAll[i].id < scoredAll[j].id
        })
        topScore := scoredAll[0].score
        var ties []string
        for _, s := range scoredAll {
            if s.score == topScore {
                ties = append(ties, s.id)
            }
        }
        if len(ties) == 1 {
            matched[name] = ties[0]
        } else {
            ambiguous = append(ambiguous, AmbiguousMatch{EnvName: name, Ca
        }
    }
    return
}
```

- [ ] **Step 4: Run â€” expect all PASS** (matching Python parity).
- [ ] **Step 5: Commit**: `feat(boba-jackett): matcher port from Python (fuzzy + override)`

Task 13: Autoconfig orchestrator (DB-aware)

Files: - Create: `qBitTorrent-go/internal/jackett/autoconfig.go` - Test: `qBitTorrent-go/internal/jackett/autoconfig_test.go`

- [ ] **Step 1: Write failing tests** covering: no credentials, missing API key, indexer requires cookie but cred has only userpass (skip with `no_compatible_credential_fields_for_indexer`), happy path with mocked Jackett.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ]

Step 3: Implement orchestrator using credentials + indexers + runs repos. The orchestrator:

1. Loads all credential bundles from `credentials` repo.
2. Loads override map from `overrides` repo merged with `JACKETT_INDEXER_MAP` env.
3. WarmUp + GetCatalog via `jackett.Client`.
4. Calls `MatchIndexers`.
5. For each match: GetIndexerTemplate â†' fill fields â†' PostIndexerConfig â†' record indexer row.
6. Inserts run summary into `autoconfig_runs`.
7. Returns `AutoconfigResult` matching the Python shape.

- [ ] **Step 4: Run â€" expect PASS**

- [ ] **Step 5: Commit**: `feat(boba-jackett): autoconfig orchestrator (DB-aware port)`

Task 14: Logging redactor

Files: - Create: `qBitTorrent-go/internal/logging/redactor.go` - Test: `qBitTorrent-go/internal/logging/redactor_test.go`

- [ ] **Step 1: Write failing test**

```
package logging

import (
    "bytes"
    "strings"
    "testing"
)

func TestRedactorReplacesSecrets(t *testing.T) {
    var buf bytes.Buffer
    r := NewRedactor(&buf)
    r.AddSecret("supersecret123")
    r.Write([]byte("user logged in with password=supersecret123 ok"))
    if strings.Contains(buf.String(), "supersecret123") {
        t.Fatalf("secret leaked: %s", buf.String())
    }
    if !strings.Contains(buf.String(), "***") {
        t.Fatalf("no redaction marker: %s", buf.String())
    }
}

func TestRedactorMultipleSecrets(t *testing.T) {
    var buf bytes.Buffer
    r := NewRedactor(&buf)
    r.AddSecret("aaa")
```

```
    r.AddSecret("bbb")
    r.Write([]byte("aaa bbb ccc"))
    if strings.Contains(buf.String(), "aaa") || strings.Contains(buf.Strin
        t.Fatalf("secret leaked: %s", buf.String())
    }
}
```

- ☐ **Step 2: Run â€” expect FAIL**
- ☐ **Step 3: Implement**

```
package logging

import (
    "bytes"
    "io"
    "sync"
)

type Redactor struct {
    dest    io.Writer
    mu      sync.RWMutex
    secrets [][]byte
    mask    []byte
}

func NewRedactor(dest io.Writer) *Redactor {
    return &Redactor{dest: dest, mask: []byte("***")}
}

func (r *Redactor) AddSecret(s string) {
    if s == "" {
        return
    }
    r.mu.Lock()
    defer r.mu.Unlock()
    r.secrets = append(r.secrets, []byte(s))
}

func (r *Redactor) RemoveSecret(s string) {
    if s == "" {
        return
    }
    r.mu.Lock()
    defer r.mu.Unlock()
    target := []byte(s)
    for i, sec := range r.secrets {
        if bytes.Equal(sec, target) {
            r.secrets = append(r.secrets[:i], r.secrets[i+1:]...)
            return
        }
    }
```

```
}

func (r *Redactor) Write(p []byte) (int, error) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    out := p
    for _, sec := range r.secrets {
        out = bytes.ReplaceAll(out, sec, r.mask)
    }
    _, err := r.dest.Write(out)
    return len(p), err
}
```

- [ ] **Step 4: Run â€” expect 2 PASS**
- [ ] **Step 5: Commit**: `feat(boba-jackett): logging redactor for credential leak prevention`

Phase 3 â€” HTTP API (Tasks 15-22)

Task 15: Auth middleware (admin/admin session)

Files: - Create: `qBitTorrent-go/internal/jackettapi/auth_middleware.go` - Test: `qBitTorrent-go/internal/jackettapi/auth_middleware_test.go`

- [ ] **Step 1: Write failing test** that asserts: GET requests pass without auth, mutating requests (POST/PATCH/DELETE) without admin/admin Basic auth header â†’ 401.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement** middleware that wraps `http.Handler`, checks `Authorization: Basic base64("admin:admin")` for non-GET methods. (Matches the projectâ€™s hardcoded creds per CLAUDE.md.)
- [ ] **Step 4: Run â€” expect PASS**
- [ ] **Step 5: Commit**: `feat(boba-jackett): auth middleware (admin/admin Basic for mutations)`

Task 16: Credentials API endpoints

Files: - Create: `qBitTorrent-go/internal/jackettapi/credentials.go` - Test: `qBitTorrent-go/internal/jackettapi/credentials_test.go`

- [ ] **Step 1: Write failing tests** for each endpoint in spec Â§8.1 using `httptest.NewServer` over the `Credentials` repo. Verify GET response shape NEVER contains plaintext values; POST adds row; DELETE removes; PATCH semantics.
-

- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement handlers** with the JSON shapes from Â§8.1. Each handler also writes to `.env` via `envfile.Upsert/Delete` AFTER the DB write succeeds. Failure of `.env` write logs warning but doesnâ€™t fail the request (DB is canonical).

Wait â€” the spec demands hybrid C: BOTH writes must succeed. Re-read spec Â§7. Actually Â§7 says dashboard writes â€œ… `encrypted` and â€œ… `atomic mirror` â€ both required. So if `.env` write fails, we MUST roll back the DB. Implement transactional pattern:

```
1. Begin tx
2. DB upsert
3. .env upsert (atomic â€” file-level)
4. If .env fails: tx rollback + return 500
5. tx commit
```

Also: trigger autoconfig replay for that single tracker after success (spec Â§7 row 1).

- [ ] **Step 4: Run â€” expect PASS**
- [ ] **Step 5: Commit**: `feat(boba-jackett): credentials API with hybrid DB+env write`

Task 17: Indexers API endpoints

Files: - Create: `qBitTorrent-go/internal/jackettapi/indexers.go` - Test: `qBitTorrent-go/internal/jackettapi/indexers_test.go`

- [ ] **Step 1: Write failing tests** for spec Â§8.2 endpoints: list, configure (POST), delete, test, toggle.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement** handlers. POST /indexers/{id} fetches the template from Jackett, populates with credentialâ€™s username/password/cookies (via Credentials repo Get), POSTs to Jackett, upserts the indexers row. DELETE removes from Jackett + DB.
- [ ] **Step 4: Run â€” expect PASS**
- [ ] **Step 5: Commit**: `feat(boba-jackett): indexers API (configure/remove/test/toggle)`

Task 18: Catalog API endpoints

Files: - Create: `qBitTorrent-go/internal/jackettapi/catalog.go` - Test: `qBitTorrent-go/internal/jackettapi/catalog_test.go`

- [ ] **Step 1: Write failing tests** for paginated query + force-refresh.
- [ ]

Step 2: Run â€” expect FAIL

- [ ] **Step 3: Implement** GET /catalog with pagination + filters; POST /catalog/refresh re-fetches from Jackett and upserts all 620 rows.
- [ ] **Step 4: Run â€” expect PASS**
- [ ] **Step 5: Commit**: `feat(boba-jackett): catalog API (paginated browse + refresh)`

Task 19: Autoconfig runs API + manual trigger

Files: - Create: `qBitTorrent-go/internal/jackettapi/runs.go` - Test: `qBitTorrent-go/internal/jackettapi/runs_test.go`

- [ ] **Step 1: Write failing tests** for list/get/run.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement** GET /autoconfig/runs (list with limit), GET /autoconfig/runs/{id}, POST /autoconfig/run (synchronous run).
- [ ] **Step 4: Run â€” expect PASS**
- [ ] **Step 5: Commit**: `feat(boba-jackett): autoconfig runs history + manual trigger API`

Task 20: Overrides API + Health endpoint

Files: - Create: `qBitTorrent-go/internal/jackettapi/overrides.go` - Create: `qBitTorrent-go/internal/jackettapi/health.go` - Test: `qBitTorrent-go/internal/jackettapi/overrides_test.go` - Test: `qBitTorrent-go/internal/jackettapi/health_test.go`

- [ ] **Step 1: Write failing tests** â€” overrides CRUD + healthz returns `{status, db_ok, jackett_ok, version, uptime_s}`.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement** both. Health pings DB (`SELECT 1`) and Jackett (`GET /health`).
- [ ] **Step 4: Run â€” expect PASS**
- [ ] **Step 5: Commit**: `feat(boba-jackett): overrides API + healthz`

Task 21: Router + redactor wiring + main.go

Files: - Create: `qBitTorrent-go/internal/jackettapi/router.go` - Modify: `qBitTorrent-go/cmd/boba-jackett/main.go`

- [ ] **Step 1: Implement router**

```
package jackettapi

import (
    "net/http"
)

type Deps struct {
    // repos, clients, autoconfig orchestrator, redactor
}

func NewMux(d *Deps) http.Handler {
    mux := http.NewServeMux()
    // register all handlers from Tasks 16-20
    return WithAuth(d, mux)
}
```

- [ ] **Step 2: Wire** `cmd/boba-jackett/main.go` to:
    1. Read env (BOBA_DB_PATH, BOBA_ENV_PATH, JACKETT_URL, JACKETT_API_KEY, PORT).
    2. EnsureMasterKey.
    3. Open DB + Migrate.
    4. Construct repos, Jackett client, redactor (set as os.Stdout/Stderr writer; populate from credentials.List() decrypted values once on boot).
    5. Run first-pass autoconfig.
    6. Start HTTP server on PORT.
- [ ] **Step 3: Build + run smoke**

```
cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 bash scripts/build.sh
BOBA_DB_PATH=/tmp/test-boba.db BOBA_ENV_PATH=/tmp/test.env JACKETT_URL=htt
sleep 3
curl -sf http://localhost:7189/healthz && echo OK
kill %1
```

Expected: `OK` printed.

- [ ] **Step 4: Commit**: `feat(boba-jackett): router + main wiring + boot smoke verified`

Task 22: OpenAPI 3.1 spec generation

Files: - Create: `qBitTorrent-go/internal/jackettapi/openapi.go`

- [ ] **Step 1: Write OpenAPI spec by hand** as a Go string constant (or embed `openapi.json` from disk). Cover every endpoint from Â§8 with request/response schemas matching the implemented handlers.
- [ ] **Step 2: Add `GET /openapi.json` handler** that serves it.
- [ ] **Step 3: Smoke test**: `curl http://localhost:7189/openapi.json | jq '.paths | keys | length'` â€" expect 14+ paths.
- [ ] **Step 4: Commit**: `feat(boba-jackett): OpenAPI 3.1 spec at /openapi.json`

Phase 4 â€" Container integration (Tasks 23-25)

Task 23: Dockerfile.jackett

Files: - Create: `qBitTorrent-go/Dockerfile.jackett`

- [ ] **Step 1: Write multi-stage Dockerfile**

```
FROM golang:1.26-alpine AS build
RUN apk add --no-cache build-base sqlite-dev
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=1 go build -o /out/boba-jackett ./cmd/boba-jackett

FROM alpine:3.20
RUN apk add --no-cache ca-certificates sqlite-libs wget
COPY --from=build /out/boba-jackett /usr/local/bin/boba-jackett
EXPOSE 7189
ENTRYPOINT ["/usr/local/bin/boba-jackett"]
```

- [ ] **Step 2: Build image manually**: `cd qBitTorrent-go && nice -n 19 podman build -f Dockerfile.jackett -t boba-jackett:dev .` Expected: image built.
- [ ] **Step 3: Commit**: `feat(boba-jackett): multi-stage Dockerfile`

Task 24: docker-compose.yml service

Files: - Modify: `docker-compose.yml`

- [ ] **Step 1: Append the `boba-jackett:` service block from spec Â§4.2 verbatim.**
- [ ] **Step 2: Validate compose**: `podman-compose config 2>&1 | head -30` Expected: no errors.
- [ ] **Step 3: Commit**: `feat(docker-compose): add boba-jackett service on :7189`

Task 25: start.sh ensure_boba_master_key

Files: - Modify: `start.sh`

- [ ] **Step 1: Add function**

```
ensure_boba_master_key() {
    local env_file="$SCRIPT_DIR/.env"
    [[ ! -f "$env_file" ]] && touch "$env_file" && chmod 0600 "$env_file"
    if grep -qE '^BOBA_MASTER_KEY=[0-9a-fA-F]{64}$' "$env_file"; then
        return 0
    fi
    print_info "Generating BOBA_MASTER_KEY..."
    local key
    key=$(head -c 32 /dev/urandom | xxd -p -c 64)
    {
        echo ""
        echo "# === BOBA SYSTEM ==="
        echo "# Master key for credential encryption-at-rest in config/bob
        echo "# DO NOT LOSE THIS â€” credentials become unrecoverable with
        echo "BOBA_MASTER_KEY=$key"
    } >> "$env_file"
    chmod 0600 "$env_file"
    print_success "BOBA_MASTER_KEY persisted to .env"
}
```

- [ ] **Step 2: Call `ensure_boba_master_key` in `main()` after `load_environment`.**
- [ ] **Step 3: Verify**: `bash -n start.sh && rm -f /tmp/.env-test && touch /tmp/.env-test && chmod 0600 /tmp/.env-test && SCRIPT_DIR=/tmp bash -c '. start.sh; ensure_boba_master_key' && grep BOBA_MASTER_KEY /tmp/.env-test`
- [ ] **Step 4: Commit**: `feat(start.sh): auto-generate BOBA_MASTER_KEY into .env`

Phase 5 â€” Frontend (Tasks 26-32)

These tasks add Angular routes, components, and the IPTorrents cookie capture flow. Each task includes the test step inside the Angular spec runner.

Task 26: Angular routes + nav + service

Files: - Modify: `frontend/src/app/app.routes.ts` - Modify: `frontend/src/app/app.html` (add nav entry) - Create: `frontend/src/app/jackett/jackett.module.ts` - Create: `frontend/src/app/jackett/jackett-routing.module.ts`

- [ ] **Step 1: Define lazy-loaded route**

```
// app.routes.ts addition
{
  path: 'jackett',
  loadChildren: () => import('./jackett/jackett.module').then(m => m.Jacke
}
```

- [ ] **Step 2: Add nav entry in** `app.html` following existing nav patterns.
- [ ] **Step 3: Create module + routing module** with two child routes: `credentials` (default) and `indexers`.
- [ ] **Step 4: Verify route resolves**: `cd frontend && nice -n 19 ng build --configuration production` â€” should succeed.
- [ ] **Step 5: Commit**: `feat(frontend): /jackett route scaffold`

Task 27: Credentials page + service

Files: - Create: `frontend/src/app/jackett/credentials/credentials.service.ts` - Create: `frontend/src/app/jackett/credentials/credentials.component.{ts,html,scss}` - Create: `frontend/src/app/jackett/credentials/credential-edit-dialog.component.ts`

- [ ] **Step 1: Write Vitest spec** asserting service hits `http://localhost:7189/api/v1/jackett/credentials` with proper auth header.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement service** using `HttpClient`, returning `Observable<CredentialMetadata[]>`.
- [ ] **Step 4: Implement component** with Material table, columns from spec Â§9.1.
- [ ] **Step 5: Implement edit dialog** with PATCH semantics (empty field = unchanged).
- [ ]

Step 6: Run tests + build: `cd frontend && nice -n 19 ng test --watch=false && nice -n 19 ng build`

- [ ] **Step 7: Commit**: `feat(frontend): credentials management page`

Task 28: Indexers â€” Configured tab

Files: - Create: `frontend/src/app/jackett/indexers/indexers.service.ts` - Create: `frontend/src/app/jackett/indexers/indexers.component.{ts,html,scss}` - Create: `frontend/src/app/jackett/indexers/configured-tab.component.ts`

- [ ] **Step 1: Vitest spec** for service GET /indexers shape + Configured tab rendering.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement service + tab container component (Material tabs).**
- [ ] **Step 4: Implement Configured tab** â€” Material table with status badges, enable toggle, test/remove actions.
- [ ] **Step 5: Tests + build pass**.
- [ ] **Step 6: Commit**: `feat(frontend): indexers Configured tab`

Task 29: Indexers â€” Browse Catalog tab

Files: - Create: `frontend/src/app/jackett/indexers/catalog-tab.component.ts` - Create: `frontend/src/app/jackett/indexers/indexer-add-dialog.component.ts`

- [ ] **Step 1: Vitest spec** for paginated catalog table + add modal opens with pre-populated fields.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement Catalog tab** with search + type/language filters + 50/page server-side pagination.
- [ ] **Step 4: Implement add dialog** â€” pulls indexer template from `GET /indexers/{id}/template-fields` (NOTE: this endpoint must exist in the API; if missing, add it as a sub-task to Â§8.2 and Task 17 retroactively â€” flag as Task 17-bis).

[Plan note: Add Task 17-bis if Task 29 surfaces this gap. The current Â§8 spec includes POST /indexers/{id} but doesnâ€™t expose the template separately; the Add dialog needs the template fields to render. Add `GET /indexers/{id}/template` to Task 17 if not already present.]

- [ ] **Step 5: Tests + build pass.**
- [ ]

Step 6: Commit: `feat(frontend): indexers Browse Catalog tab + add dialog`

Task 30: Indexers â€” Autoconfig History tab

Files: - Create: `frontend/src/app/jackett/indexers/history-tab.component.ts`

- [ ] **Step 1: Vitest spec** â€” list of last 50 runs, click expands to show JSON viewer.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement** with `ngx-json-viewer` (add dep) or similar; â€œRun autoconfig nowâ€¦ button calls `POST /autoconfig/run`.
- [ ] **Step 4: Tests + build pass.**
- [ ] **Step 5: Commit**: `feat(frontend): indexers Autoconfig History tab`

Task 31: IPTorrents cookie capture flow

Files: - Create: `frontend/src/app/jackett/indexers/iptorrents-cookie-flow.component.ts`

- [ ] **Step 1: Vitest spec** â€” when add dialog opens for an indexer with cookie-only required fields, render the instruction panel + textarea + save button.
- [ ] **Step 2: Run â€” expect FAIL**
- [ ] **Step 3: Implement** â€” detect cookie-only template via `template_fields` filter; render instruction panel from spec Â§9.3 verbatim; on save, POST to `/credentials` (kind=cookie) then POST to `/indexers/iptorrents`.
- [ ] **Step 4: Tests + build pass.**
- [ ] **Step 5: Commit**: `feat(frontend): IPTorrents cookie capture flow`

Task 32: NNMClub clarification banner

Files: - Modify: `frontend/src/app/jackett/credentials/credentials.component.ts` - Modify: `qBitTorrent-go/internal/jackett/autoconfig.go` (add `served_by_native_plugin` flag) - Modify: `qBitTorrent-go/internal/jackett/autoconfig_test.go` (assert flag presence)

- [ ] **Step 1: Add Go field**: extend `AutoconfigResult` with `ServedByNativePlugin []string` populated by checking each `skipped_no_match` entry against a known list (`NNMCLUB`, plus any other native-plugin name from `plugins/*.py` matched lowercased).
-

- [ ] **Step 2: Update Go test** asserting NNMCLUB in skipped_no_match also appears in served_by_native_plugin.
- [ ] **Step 3: Update Vitest spec** asserting banner renders for each name in served_by_native_plugin.
- [ ] **Step 4: Run all â€” expect PASS.**
- [ ] **Step 5: Commit**: `feat(jackett): NNMClub clarification â€” served_by_native_plugin field + UI banner`

Phase 6 â€” Higher test layers (Tasks 33-37)

Task 33: Layer 2 integration tests

Files: - Create: `qBitTorrent-go/tests/integration/jackett_db_test.go`

- [ ] **Step 1: Write tests** covering the 6 scenarios from spec Â§10.2 (bootstrap from empty .env, bootstrap with N triples, restart no re-import, UI add cred â†’ DB+env consistent, UI delete â†’ both gone, concurrent dashboard writes).
- [ ] **Step 2: Run with build tag**: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -tags=integration ./tests/integration/ -v` Expected: all PASS.
- [ ] **Step 3: Commit**: `test(boba-jackett): Layer 2 integration tests against real SQLite + .env`

Task 34: Layer 3 E2E tests (real Jackett)

Files: - Create: `qBitTorrent-go/tests/e2e/jackett_management_test.go`

Pre-requisite: stack is up (`./start.sh -p`).

- [ ] **Step 1: Write tests** for the 7 scenarios from spec Â§10.3.
- [ ] **Step 2: Run**: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -tags=e2e ./tests/e2e/ -v` Expected: all PASS (tests skip cleanly if Jackett unreachable per CONST-11).
- [ ] **Step 3: Commit**: `test(boba-jackett): Layer 3 E2E against live stack`

Task 35: Layer 4 security/leak tests + bandit/gosec

Files: - Create: `qBitTorrent-go/tests/security/credential_leak_test.go` - Create: `challenges/scripts/credential_leak_grep_challenge.sh`

- [ ] **Step 1: Write the leak test** per spec Â§10.4 â€" insert 6 high-entropy creds, drive E2E, capture all logs+responses+/proc env, grep for literal values, assert 0 hits.
- [ ] **Step 2: Write the challenge wrapper.**
- [ ] **Step 3: Run**: `bash challenges/scripts/credential_leak_grep_challenge.sh` Expected: PASS.
- [ ] **Step 4: Run security scanners**:

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 gosec ./qBitTorrent-go/...
GOMAXPROCS=2 nice -n 19 ionice -c 3 bandit -r download-proxy/src/ -ll
```

Expected: no high-severity issues (or all addressed/justified).

- [ ] **Step 5: File permission check** â€" bash script asserts `config/boba.db` and `.env` are mode `0600`. Add as `challenges/scripts/boba_db_file_perms_challenge.sh`.
- [ ] **Step 6: Commit**: `test(boba-jackett): Layer 4 security (leak grep + gosec/bandit + file perms)`

Task 36: Layer 5 benchmarks

Files: - Create: `qBitTorrent-go/internal/db/repos/catalog_bench_test.go` - Create: `qBitTorrent-go/internal/db/crypto_bench_test.go` - Create: `qBitTorrent-go/internal/jackett/autoconfig_bench_test.go`

- [ ] **Step 1: Write benchmarks** per spec Â§10.5 â€" three scenarios for catalog query, encrypt/decrypt, full autoconfig run.
- [ ] **Step 2: Run**: `cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -bench=. -benchmem -benchtime=3s ./internal/db/repos/ ./internal/db/ ./internal/jackett/ -run=^$` Expected: all latency thresholds from Â§10.5 met.
- [ ] **Step 3: Commit**: `test(boba-jackett): Layer 5 benchmarks (catalog query, crypto, autoconfig)`

Task 37: Layer 6 contract tests + Layer 7 challenges

Files: - Create: `qBitTorrent-go/tests/contract/openapi_test.go` - Create: `challenges/scripts/cred_roundtrip_challenge.sh` - Create: `challenges/scripts/env_db_drift_challenge.sh` - Create: `challenges/scripts/`

`iptorrents_cookie_flow_challenge.sh` - Create: `challenges/scripts/`
`master_key_autogen_challenge.sh` - Create: `challenges/scripts/`
`nnmclub_native_plugin_clarification_challenge.sh`

- [ ] **Step 1: Contract test** loads `openapi.json`, hits every endpoint, validates response shape using `getkin/kin-openapi`.
- [ ] **Step 2: Write all 5 challenge scripts** following the projectâ€™s existing challenge pattern (see `challenges/scripts/no_suspend_calls_challenge.sh` as reference for structure).
- [ ] **Step 3: Add all to** `challenges/scripts/run_all_challenges.sh` (the file is currently untracked from the initial state â€” Task 38 will commit it; add an entry â€œTODO: registerâ€¦ if it doesnâ€™t exist yet).

Wait â€” `run_all_challenges.sh` is in the initial git status as untracked. Let me handle: if not tracked, add it in this task with the new challenges registered + the existing ones.

```
[[ ! -f challenges/scripts/run_all_challenges.sh ]] && cat > challenges/sc
#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
fail=0
for script in "$SCRIPT_DIR"/*_challenge.sh; do
    echo "=== Running $(basename "$script") ==="
    if ! bash "$script"; then
        fail=$((fail+1))
        echo "FAIL: $(basename "$script")"
    fi
done
if [[ $fail -gt 0 ]]; then
    echo "$fail challenge(s) failed"
    exit 1
fi
echo "All challenges passed"
EOF
chmod +x challenges/scripts/run_all_challenges.sh
```

- [ ] **Step 4: Run**: `bash challenges/scripts/run_all_challenges.sh` Expected: all PASS.
- [ ] **Step 5: Commit**: `test(boba-jackett): Layer 6 contract + Layer 7 challenge suite`

Phase 7 â€” Cleanup + Definition of Done (Tasks 38-46)

Task 38: Remove Python /api/v1/jackett/autoconfig/last endpoint

Files: - Modify: `download-proxy/src/api/__init__.py` (remove jackett router include) - Delete: `download-proxy/src/api/jackett.py` - Modify: `download-proxy/src/merge_service/jackett_autoconfig.py` (mark deprecated; keep for now â€” Phase 2 removes the module entirely)

- [ ] **Step 1: Verify no other code imports** `download_proxy.src.api.jackett`: `grep -rn 'from.*api.jackett' download-proxy/src/`
- [ ] **Step 2: Remove the include from** `__init__.py` + delete the file.
- [ ] **Step 3: Run Python unit tests**: `python3 -m pytest tests/unit/ -v --import-mode=importlib` Expected: any tests that imported the removed endpoint now fail; remove or update them.
- [ ] **Step 4: Verify python proxy still boots cleanly via cache-bust + restart per CLAUDE.md.**
- [ ] **Step 5: Commit**: `refactor(python-proxy): remove /api/v1/jackett/autoconfig/last (replaced by Go service)`

Task 39: Reconcile predecessor plan checkboxes (DoD #4)

Files: - Modify: `docs/superpowers/plans/2026-04-26-jackett-autoconfig-clean-rebuild.md`

- [ ] **Step 1: For each unchecked** `- [ ]` **in the predecessor plan, verify against shipped commits** (`git log --all --grep=jackett_autoconfig --oneline`) and the actual code in `download-proxy/src/merge_service/jackett_autoconfig.py`.
- [ ] **Step 2: Mark** `- [x]` **for every step that maps to shipped code** (should be most of Layers 1-7).
- [ ] **Step 3: For any item still genuinely undone, add a note** explaining why it was deferred.
- [ ] **Step 4: Commit**: `docs(plan): reconcile 2026-04-26 jackett-autoconfig plan checkboxes against shipped commits`

Task 40: Documentation â€” JACKETT_INTEGRATION.md + CLAUDE.md + AGENTS.md

Files: - Modify: `docs/JACKETT_INTEGRATION.md` (rewrite â€œAuto-Configurationâ€ section as DB-backed) - Modify: `CLAUDE.md` (Architecture section + Port Map + Env Vars adds BOBA_MASTER_KEY, BOBA_DB_PATH) - Modify: `AGENTS.md` (mirror CLAUDE.md updates)

- [ ]

Step 1: Update JACKETT_INTEGRATION.md â€" section â€œAuto-Configurationâ€ now describes the DB layer + UI as canonical; .env import as bootstrap-only.

- [ ] **Step 2: Update CLAUDE.md** Port Map: add row `| 7189 | boba-jackett (Go) | http://localhost:7189 |`.
- [ ] **Step 3: Update CLAUDE.md** Architecture section to list `boba-jackett` as the 4th container.
- [ ] **Step 4: Update CLAUDE.md** Environment Variables section: `BOBA_MASTER_KEY` (auto-generated 32-byte hex), `BOBA_DB_PATH` (defaults `/config/boba.db`).
- [ ] **Step 5: Mirror all three updates into AGENTS.md.**
- [ ] **Step 6: Commit**: `docs: CLAUDE.md + AGENTS.md + JACKETT_INTEGRATION.md updates for boba-jackett + DB`

Task 41: Documentation â€" BOBA_DATABASE.md (NEW)

Files: - Create: `docs/BOBA_DATABASE.md`

- [ ] **Step 1: Write doc** covering: schema reference (link to spec Â§5), master key lifecycle (auto-gen + manual rotation procedure: stop service, decrypt all rows with old key, re-encrypt with new, update .env), backup procedure (`podman exec boba-jackett wget -qO- http://localhost:7189/api/v1/admin/backup` OR simply `cp config/boba.db config/boba.db.bak`), recovery from lost key (no recovery â€" re-bootstrap from .env if all triples are still there).
- [ ] **Step 2: Commit**: `docs: add BOBA_DATABASE.md (schema/key lifecycle/backup/recovery)`

Task 42: Bugfix doc

Files: - Create OR modify: `docs/issues/fixed/BUGFIXES.md`

- [ ] **Step 1: For every bug surfaced + fixed during the implementation of Tasks 1-41, append an entry** with: title, root cause, affected files, fix description, link to test/challenge that prevents regression.
- [ ] **Step 2: At minimum, document the IPTorrents cookie field gap** (it was a pre-existing issue surfaced in this work â€" CONST-32 reproduction-before-fix means the iptorrents_cookie_flow_challenge.sh script reproduces it; the fix is the cookie capture flow + UI hint).
- [ ] **Step 3: Commit**: `docs(bugfixes): document IPTorrents cookie gap + any others surfaced during implementation`

Task 43: TODO/FIXME sweep + CONST-013 audit

- ☐ **Step 1: Run grep**: `grep -rE 'TODO|FIXME|XXX|HACK' qBitTorrent-go/cmd/boba-jackett qBitTorrent-go/internal/db qBitTorrent-go/internal/envfile qBitTorrent-go/internal/bootstrap qBitTorrent-go/internal/jackett qBitTorrent-go/internal/jackettapi qBitTorrent-go/internal/logging frontend/src/app/jackett docs/BOBA_DATABASE.md` Expected: 0 hits in NEW code (pre-existing entries elsewhere are out of scope).
- ☐ **Step 2: If any hits, fix them in this task** (donâ€™t defer).
- ☐ **Step 3: CONST-013 audit**: `grep -rnE '\bsync\.(Mutex|RWMutex)\b' qBitTorrent-go/cmd/boba-jackett qBitTorrent-go/internal/db qBitTorrent-go/internal/envfile qBitTorrent-go/internal/bootstrap qBitTorrent-go/internal/jackett qBitTorrent-go/internal/jackettapi qBitTorrent-go/internal/logging` For each hit, verify it does NOT guard a mutable map/slice (they should be small-state guards like the envfile writer mutex). If any guards a map/slice, refactor to `safe.Store`/`safe.Slice`.
- ☐ **Step 4: Commit if anything changed**: `fix(boba-jackett): TODO/FIXME sweep + CONST-013 audit`

Task 44: CONST-033 verification

- ☐ **Step 1: Run both challenges**:

```
bash challenges/scripts/no_suspend_calls_challenge.sh
bash challenges/scripts/host_no_auto_suspend_challenge.sh
```

Expected: both PASS. - [ ] **Step 2: If either fails, halt and fix the source** before proceeding (per CLAUDE.md: â€œfix the source â€” do NOT extend the allowlistâ€¦). - [ ] **Step 3: No commit needed if no changes.**

Task 45: Final integration smoke (everything together)

- ☐ **Step 1: Tear down stack**: `./stop.sh --purge`
- ☐ **Step 2: Wipe DB only** (not .env): `podman unshare rm -f config/boba.db config/boba.db-wal config/boba.db-shm`
- ☐ **Step 3: Boot stack**: `./start.sh -p`
- ☐ **Step 4: Verify all services** (extend the prior verification with port 7189):

```
for port in 7185 7186 7187 7189 9117; do
  echo -n "  $port: "; curl -s -o /dev/null -w "HTTP %{http_code}\n" http:
done
```

Expected: all 200. - [ ] **Step 5: Verify boba.db created + master key was either pre-existing or auto-generated**: `ls -la config/boba.db && grep BOBA_MASTER_KEY .env | head -1 | sed 's/=.*/=<redacted>/'` - [ ] **Step 6: Verify autoconfig ran**: `curl -s http://localhost:7189/api/v1/jackett/autoconfig/runs?limit=1 | jq .` - [ ] **Step 7: Browse to dashboard**, navigate to `/jackett/credentials` and `/jackett/indexers`, verify all three tabs render. - [ ] **Step 8: No commit** â€" verification only.

Task 46: Pre-test commit + push to all upstreams (DoD #9)

Pre-conditions: Every implementation commit from Tasks 1-44 already exists locally.

- ☐ **Step 1: Final tree check**: `git status --short` â€" should be clean (or only the bin/ binaries which are gitignored).
- ☐ **Step 2: Run full test suite locally one final time**:

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 go test -short -race ./qBitTorrent-go/
cd qBitTorrent-go && GOMAXPROCS=2 nice -n 19 go test -tags=integration ./t
GOMAXPROCS=2 nice -n 19 go test -tags=e2e ./tests/e2e/ -v
bash challenges/scripts/run_all_challenges.sh
./ci.sh --quick
```

- ☐ **Step 3: Push to every configured remote**:

```
for remote in origin github upstream; do
  echo "=== Pushing to $remote ==="
  git push $remote main
done
```

Expected: each push reports success or â€œEverything up-to-dateâ€�.

- ☐ **Step 4: No further commits in this task.**

Task 47: Definition-of-Done sweep + final run

The final gate. Execute exactly once when Tasks 1-46 are complete.

- ☐ **Step 1: Walk DoD Â§11 from spec one item at a time** and tick off each:
  - ☐ Â§11.1 â€" every endpoint in Â§8 implemented + Layer 2/3 tests pass
  - ☐ Â§11.2 â€" both UI pages work in browser (manual Playwright walkthrough recorded)
  - ☐ Â§11.3 â€" all 7 test layers green (paste output)
  - ☐ Â§11.4 â€" predecessor plan checkboxes reconciled (Task 39)
  - ☐

  - Â§11.5 â€” docs updated (Task 40+41)
  - Â§11.6 â€” bugfix doc complete (Task 42)
  - Â§11.7 â€” IPTORRENTS_COOKIES documented + UI hint
  - Â§11.8 â€” NNMClub clarification in autoconfig + UI banner
  - Â§11.9 â€” pre-test commit + push to all 3 remotes (Task 46)
  - Â§11.10 â€” demo block prepared for PR body
  - Â§11.11 â€” TODO/FIXME sweep clean (Task 43)
  - Â§11.12 â€” CONST-013 audit clean (Task 43)
  - Â§11.13 â€” CONST-033 verification both PASS (Task 44)
- **Step 2: Final demo block construction** â€” paste the output of Task 45 verification + Task 46 final test run + Task 46 push output into a `## Demo` markdown block. Save as `docs/demos/2026-04-27-boba-jackett-ui-and-db-demo.md`.
- **Step 3: Commit demo doc**:

```
git add docs/demos/2026-04-27-boba-jackett-ui-and-db-demo.md
git commit -m "$(cat <<'EOF'
docs(demo): boba-jackett DoD evidence â€” all 13 items verified

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
for remote in origin github upstream; do git push $remote main; done
```

- **Step 4: Declare done.** Reply to the user with a summary of all 13 DoD items ticked + a link to the demo doc.

Self-Review (run inline after writing this plan)

1. Spec coverage: Walked spec Â§1-Â§14. Each requirement maps to a task: - Â§4 Architecture â†’ Tasks 1, 21, 23, 24 - Â§5 Schema â†’ Task 3 - Â§5.1 Crypto envelope â†’ Task 2 - Â§5.2 File perms â†’ Task 35 - Â§6 Bootstrap flow â†’ Tasks 5, 10, 25 - Â§7 Sync semantics â†’ Task 16 (write-through pattern) - Â§8 API surface â†’ Tasks 15-22 - Â§9 UI â†’ Tasks 26-32 - Â§10 Test strategy â†’ Tasks 33-37 + inline TDD in 1-22 - Â§11 DoD (13 items) â†’ Tasks 38-47 each maps to a numbered DoD item - Â§12 Risks/Mitigations â†’ covered by tests (leak grep, atomic writes, key-mismatch detection in Task 21 main.go) - Â§13 Out of scope â†’ respected; merge service untouched

2. Placeholder scan: A grep for â€œTBD/TODO/FIXME/â€¦shouldâ€¦â€ in the plan body (excluding the parts that talk about the userâ€™s TODO sweep) found zero â€” but Task 29 has a â€œ[Plan note]â€ about a possible API gap. This is INTENTIONAL â€” it flags a real risk to address during implementation. Leaving as-is.

3. Type consistency: Cross-checked names â€” `Credential` struct (Task 6) used as `*Credential` return in `Get`/`List`; `Indexer` (Task 7) and `CatalogEntry` (Task 11) consistent across uses; `AutoconfigResult.ServedByNativePlugin` (Task 32) added to autoconfig.go in the same task that asserts it.

Self-review pass.

Execution Handoff

Plan complete. Two execution options:

1. **Subagent-Driven** (recommended) â€” fresh subagent per task, review between tasks, fast iteration.
2. **Inline Execution** â€” execute tasks in this session using `superpowers:executing-plans`, batch execution with checkpoints.

Which approach?